



CSS Grid 布局

第四课：间距到对齐的精细化控制

 Loading slide...

传统布局的困扰

在深入 Grid 对齐系统之前，让我们回顾传统布局方案的问题

传统 margin 方案的痛点

- 外边距折叠 (margin collapsing) 问题
- 需要为首尾元素单独处理多余边距
- 复杂的选择器 (`:first-child` , `:last-child`)
- 难以创建整齐划一的栅格系统

Grid 的解决方案

CSS Grid 引入了更优雅的**声明式**布局控制，从根本上解决这些长期困扰开发者的布局难题。

 Loading slide...

 Loading slide...

2. 容器级对齐：控制网格在容器中的位置

当网格的总尺寸小于其容器尺寸时，控制整个网格在容器内的位置

容器级对齐属性

- `justify-content`：控制网格在**行轴**（水平方向）上的对齐
- `align-content`：控制网格在**列轴**（垂直方向）上的对齐

常用对齐值

值	效果
<code>start</code>	网格在容器的起始位置对齐
<code>end</code>	网格在容器的结束位置对齐
<code>center</code>	网格在容器中居中
<code>space-around</code>	轨道均匀分布，首尾距边缘为轨道间距的一半
<code>space-between</code>	轨道均匀分布，首尾轨道紧贴容器边缘
<code>space-evenly</code>	完全均匀分布，所有间距相等

 Loading slide...

3. 项目级对齐：单元格内的精细控制

控制单个网格项目在其分配到的单元格内部的位置

容器统一定义

- `justify-items` : 控制所有项目在各自单元格内的水平对齐
- `align-items` : 控制所有项目在各自单元格内的垂直对齐

项目独立定义

- `justify-self` : 覆盖 `justify-items` , 控制单个项目的水平对齐
- `align-self` : 覆盖 `align-items` , 控制单个项目的垂直对齐

可选值

- `start` : 项目对齐到单元格的起始边缘
- `end` : 项目对齐到单元格的结束边缘
- `center` : 项目在单元格内居中
- `stretch` : (默认值) 项目拉伸填满整个单元格空间

 Loading slide...

 Loading slide...

总结

三个层次的控制

1. 轨道间距 (gap)

- 在网格轨道间创建一致的、无边缘问题的间隙
- 替代传统的 `margin` 方案

2. 网格整体对齐 (`justify-content` / `align-content`)

- 将整个网格系统在容器的富余空间内进行定位
- 控制网格作为整体的位置

3. 单元格内项目对齐 (`justify-items/self` / `align-items/self`)

- 容器级：为所有项目设定全局对齐规则
- 项目级：对特定项目进行个性化微调